

Système prédictif

Prédiction des délais des bus de Toronto

Contenu de la présentation

01

Machine
learning

02

Interface
web

03

Démo

01 - Machine Learning

Problème à résoudre

Problème : Pouvons-nous prédire le délai sur une ligne de bus en fonction de la météo et de la période de l'année ?

Nécessite :

- Données sur les délais des bus (au moins un an)
- Données météorologiques
- Données temporelles

Source des données

Délai des bus :

- IDFM, données incomplètes (Prim)
- TTC, données de 2023 (Kaggle)
 - Un délai en minutes par incident rapporté

Météo :

- Gouvernement du Canada, relevés horaires des conditions météo (Gov)
 - Station : 6158359, dans Toronto, données couvrant nos besoins

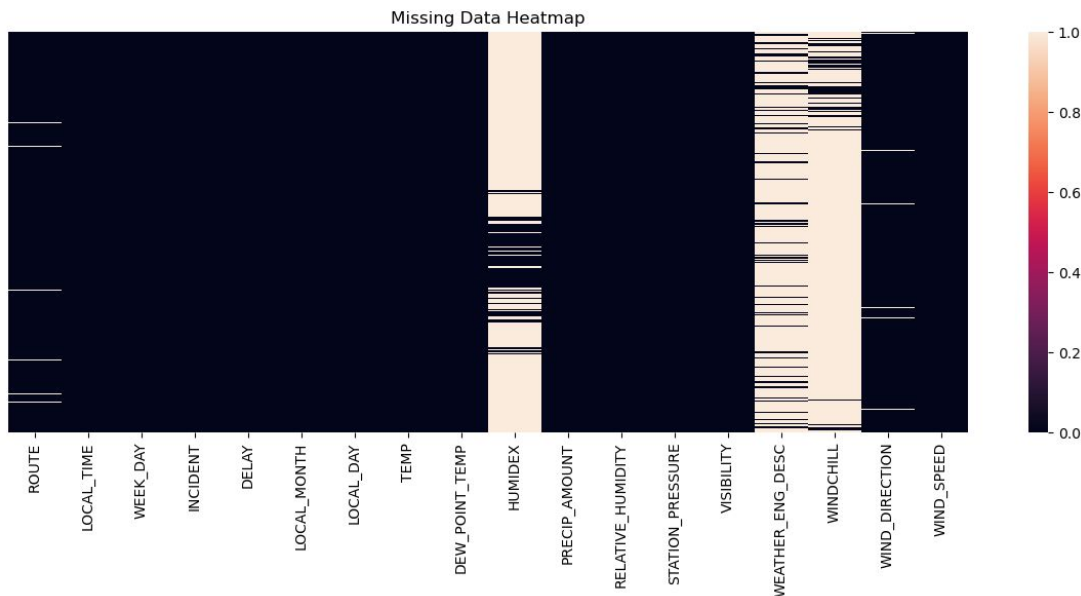
Analyse des données - Ensemble du JDD

Forme des données : 18 variables pour 50913 entrées

Données en double : 455 entrées, toutes supprimées

Données manquantes :

- ROUTE : 1.04%
- WIND_DIRECTON : 3.31%
- HUMIDEX : 78.78%
- WEATHER_ENG_DESC : 85.24%
- WINDCHILL : 89.42%



Création du jeu de données

1. A chaque incident, associer la météo (arrondi à l'heure)
2. Retirer les features inutiles
 - Métadonnées (coordonnées de la station, flags décrivant les conditions d'obtention des données, ...)
 - Duplicatas (données temporelles, ...)
 - Inexploitables (direction du bus, identifiant du bus, ...)
3. Uniformisation du nom des variables

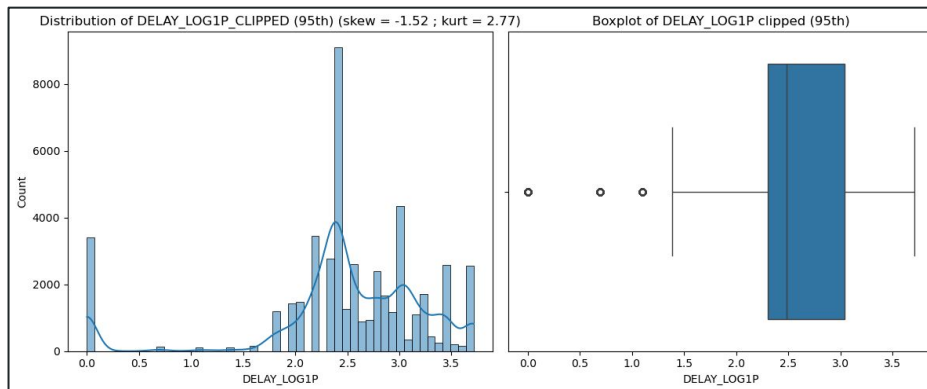
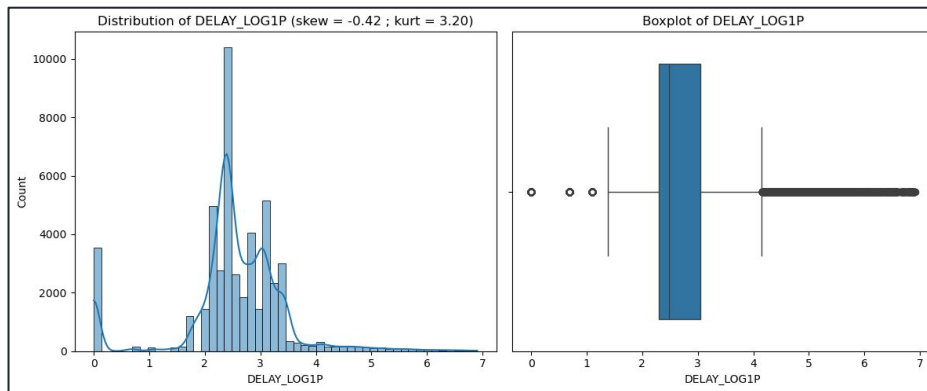
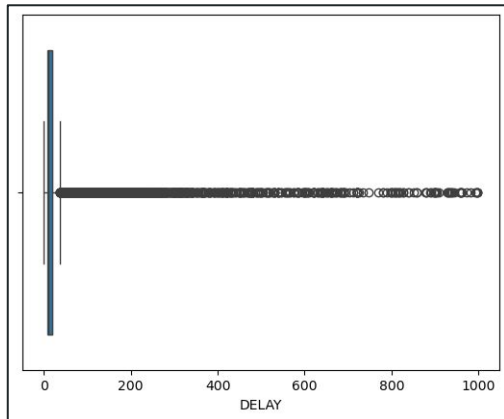
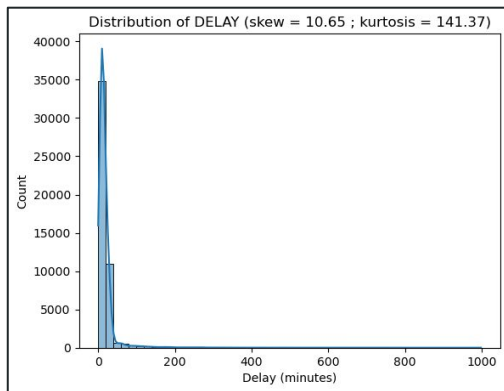
Gestion des données manquantes

Variable	Traitement	Résultat	Décision
ROUTE	Supprimer lignes	526 lignes supprimées	Conserver
WIND_DIRECTION	Supprimer lignes	1595 lignes supprimées	Conserver
HUMIDEX	Recalculer	0% de données manquantes	Conserver
WEATHER_ENG_DESC	Remplacer par "Clear"	0% de données manquantes	Conserver
WINDCHILL	Recalculer	89.23% de données manquantes	Supprimer la variable

Forme des données après le traitement :

17 variables pour 48288 entrées (-5.16% sur le jeu de données complet)

Analyse univariée - Variable cible



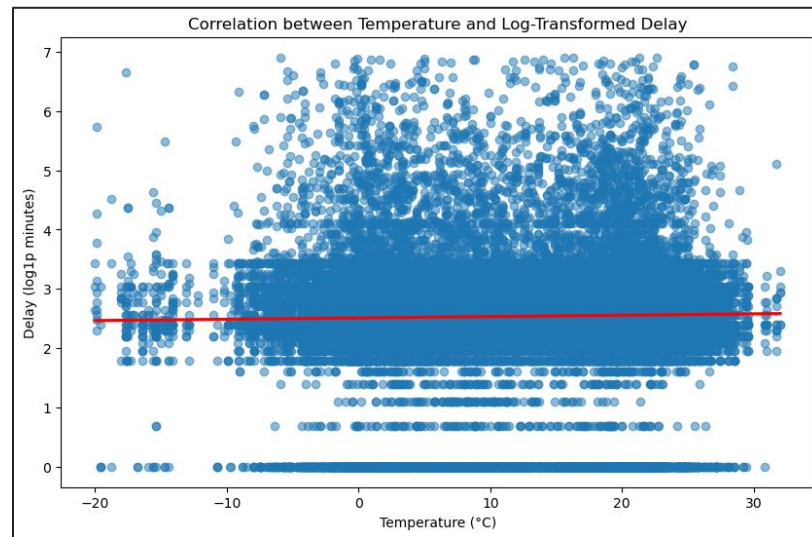
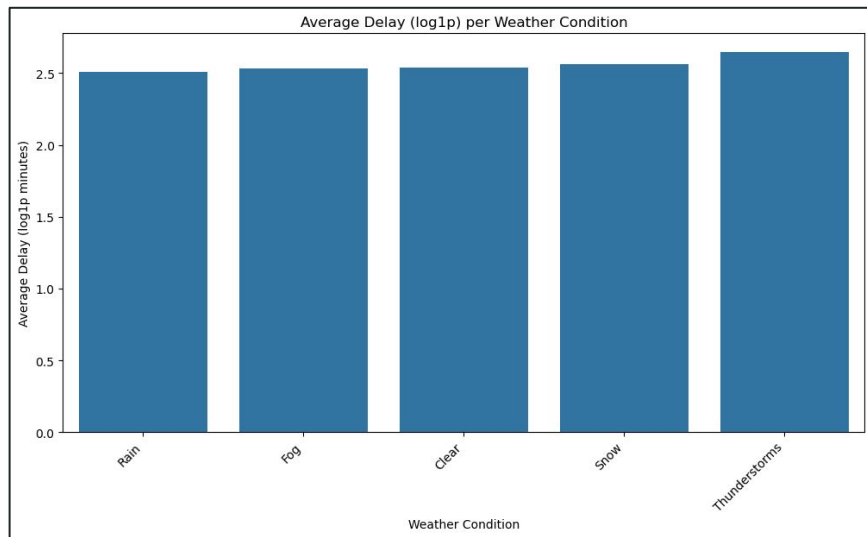
Analyse univariée - Variables explicatives

Points importants :

- Variables liées à la température → Bimodalités due aux saisons
- Routes → Données erronées supprimées (211 lignes, 0.4% des données)
- Mois → Que 2 entrées pour le mois de décembre (pareil pour 2022)
- Des variables transformées
 - Précipitations (en mm) → transformée en une variable booléenne
 - Visibilité (en km) → transformée en une variable catégorielle (très, bonne, ..., aucune visibilité)
 - Incident (13 modalités) → regroupées en 5 modalités
 - Description météo (24 modalités) → regroupées en 4 multi-modalités

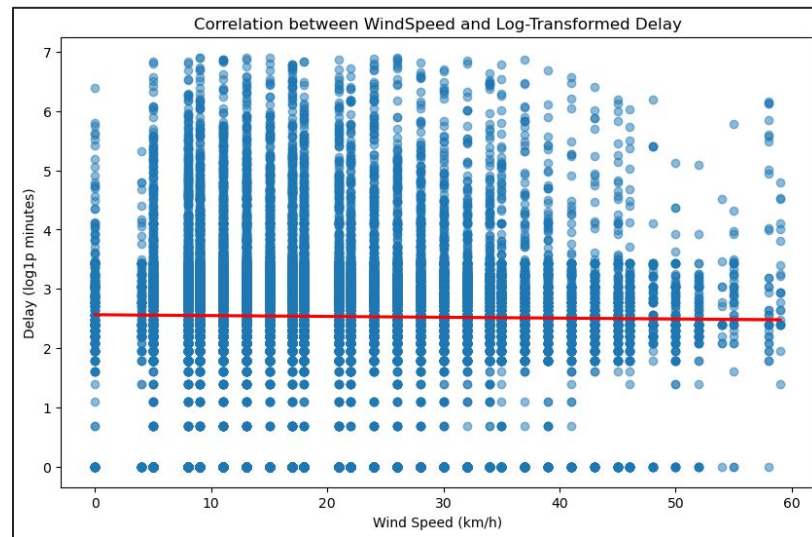
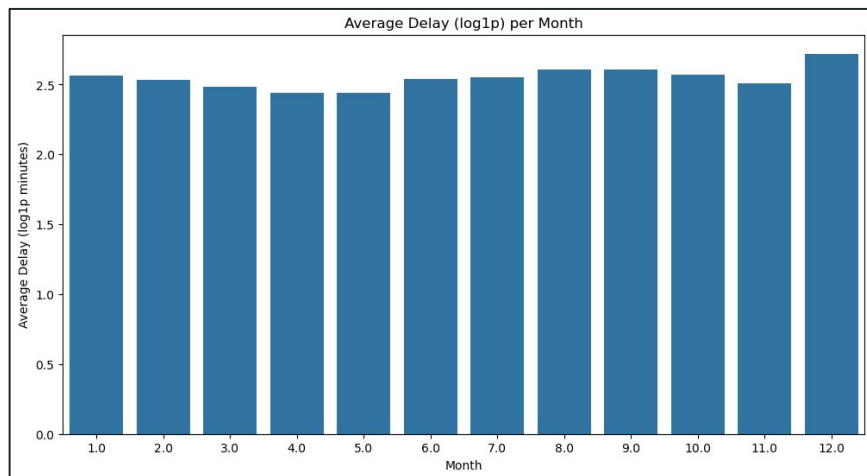
Analyse bivariable - Target vs features (1)

Problème rencontré : les données météorologiques/temporelles n'ont pas de corrélation linéaires évidentes avec la variable cible



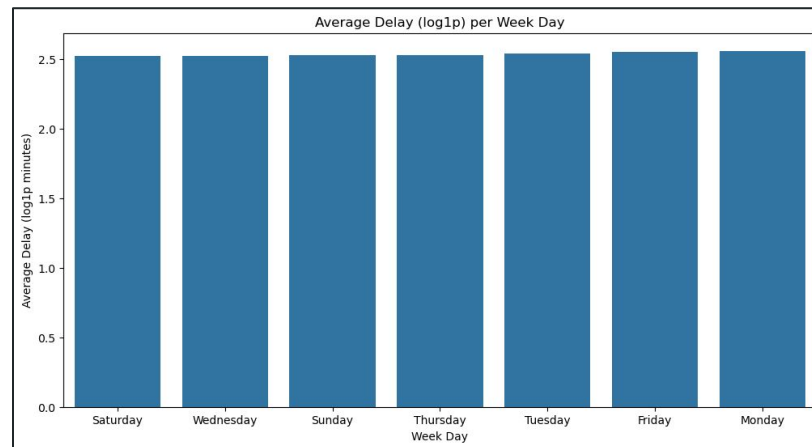
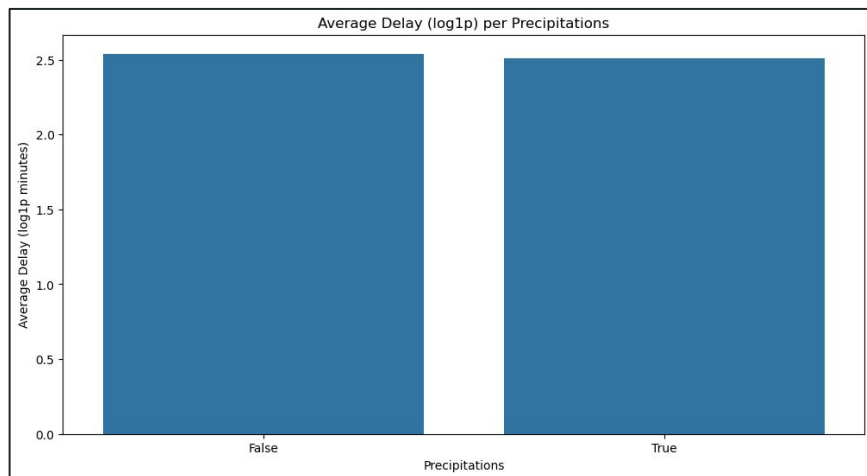
Analyse bivariable - Target vs features (2)

Problème rencontré : les données météorologiques/temporelles n'ont pas de corrélation linéaires évidentes avec la variable cible



Analyse bivariable - Target vs features (3)

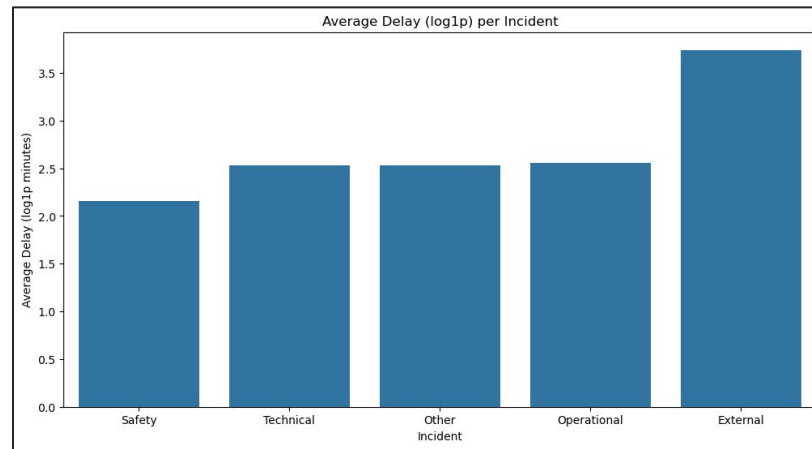
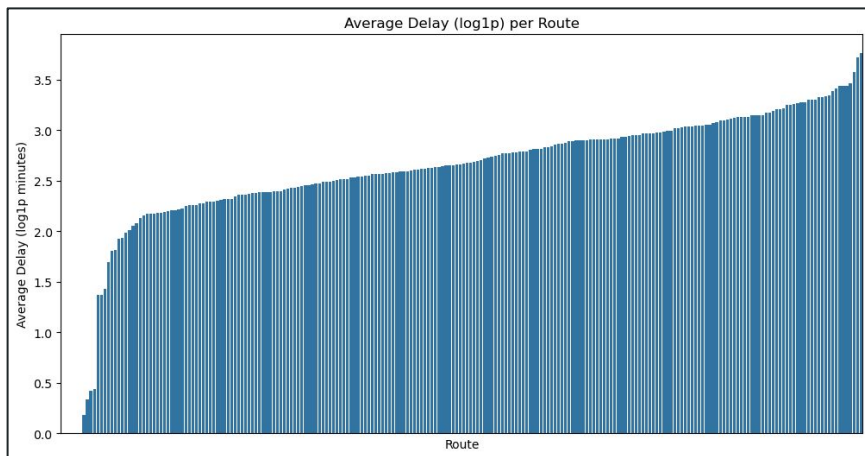
Problème rencontré : les données météorologiques/temporelles n'ont pas de corrélation linéaires évidentes avec la variable cible



Analyse bivariée - Target vs features (4)

Problème rencontré : les données météorologiques/temporelles n'ont pas de corrélation linéaires évidentes avec la variable cible

Mais : Il existe des relations avec les données liées aux incident



Entraînement de modèles - Prétraitements

En plus des modifications de variable déjà fait, voilà les changements additionnels

- **Encodage Cyclique** : heure, minute, jour de la semaine, mois, jour du mois, direction du vent
- **Transformation Yeo-Johnson** : vitesse du vent
- **Standard scaling** : température, température de rosée, humidex, humidité relative, pression atmosphérique, vitesse du vent
- **Encodage Onehot** : route, incident, saison
- **Encodage Ordinal** : visibilité

Entraînement de modèles - Scenarii

3 scenarii :

- Toutes les données
- Données d'hiver (Janvier - Avril & Octobre - Décembre)
- Données d'été (Mai - Septembre)

Cas saisonniers :

- Pas de variable de saison

Entraînement de modèles - Méthode de test

Objectif numérique continu : **Régression**

Méthodologie de test :

- Séparation Train, Test, Validation (en général, 60% / 20% / 20%)
- Hyper-optimisation sur 100 trials (en général)
- Objectif : minimiser le score RMSE
 - Pourquoi : pénalise fortement les grandes erreurs

Entraînement de modèles - Modèles testés

Modèles classiques :

- Arbre de décision
- Régression linéaire (avec et sans pénalisation)
- Random forest
- Gradient boosting
- Extreme gradient boosting
- Régression à vecteurs de support

Modèles Généralisés Additifs :

- LinearGAM
- GammaGAM
- PoissonGAM

Entraînement de modèles - Résultats

R ² ; MAE; RMSE	Toutes les saisons	Hiver	Été
Arbre de décision	0.2289 ; 0.5594 ; 0.7146	0.1957 ; 0.5746 ; 0.7482	0.1615 ; 0.5799 ; 0.7578
Régression linéaire (classique)	0.2921 ; 0.4898 ; 0.6560	0.2853 ; 0.5017 ; 0.6648	0.2826 ; 0.4824 ; 0.6484
Régression linéaire (elasticnet)	0.2948 ; 0.4902 ; 0.6535	0.2877 ; 0.5043 ; 0.6626	0.2865 ; 0.4874 ; 0.6449
Random forest	0.0140 ; 0.6220 ; 0.9137	-0.0009 ; 0.6217 ; 0.9310	-0.0002 ; 0.6112 ; 0.9040
Gradient boosting	0.3002 ; 0.4465 ; 0.6485	0.2625 ; 0.4650 ; 0.6860	0.2969 ; 0.4882 ; 0.6354
Extreme gradient boosting	0.3296 ; 0.4746 ; 0.6212	0.3182 ; 0.4916 ; 0.6342	0.3245 ; 0.4712 ; 0.6106
Régression à vecteurs de support	0.2601 ; 0.4451 ; 0.6857	0.2570 ; 0.4472 ; 0.6911	0.2053 ; 0.4885 ; 0.7183
LinearGAM	0.2958 ; 0.4896 ; 0.6526	0.3142 ; 0.4947 ; 0.6515	0.2742 ; 0.4608 ; 0.6160
GammaGAM	0.2838 ; 0.4975 ; 0.6637	0.2781 ; 0.4929 ; 0.6372	0.2746 ; 0.4928 ; 0.6622
PoissonGAM	0.2179 ; 0.5648 ; 0.7248	0.1899 ; 0.5833 ; 0.7444	0.1784 ; 0.5582 ; 0.7279

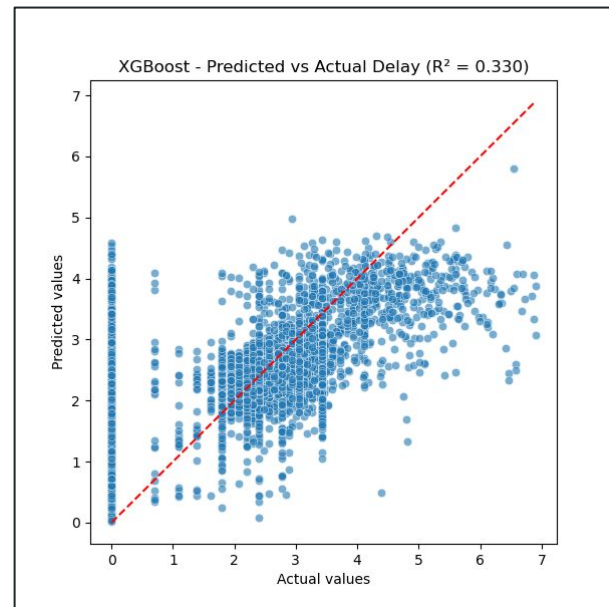
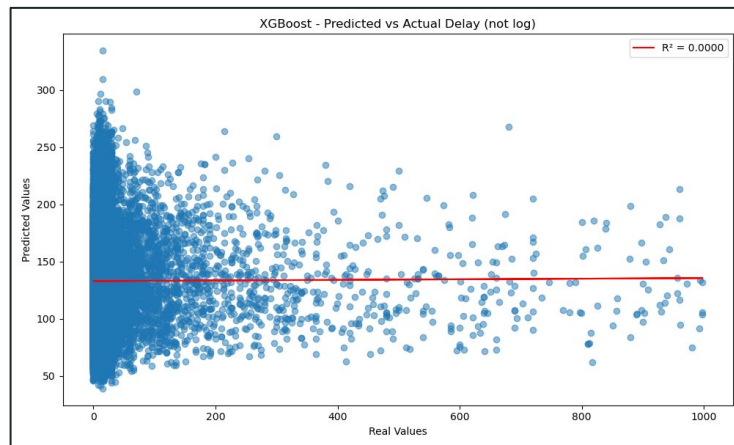
Entraînement de modèles - Résultats

Meilleur modèle : Extreme gradient boosting

Scénario le plus équilibré : Toutes les saisons

Problème : Aucun modèle n'est particulièrement bon

Point d'attention : R^2 , MAE et RMSE sont calculés sur des données logarithmique



Machine learning - Conclusion

Modèles inexploitable : Trop d'écart dans les valeurs prédites et les données réelles

Raisons possibles :

- Mauvaise gestion des outliers
- Potentiel de certaines variables inexploité
- Pas assez de features

02 - Interface Web

Introduction au projet

Objectifs:

- Visualisation interactive d'un réseau de transport urbain.
- Carte, météo en temps réel, statistiques et prédiction des retards.

Problématique

Objectifs:

- Beaucoup de données hétérogènes à afficher.
- Besoin de lisibilité et de performances sur une carte Web.
- Architecture modulaire et maintenable.

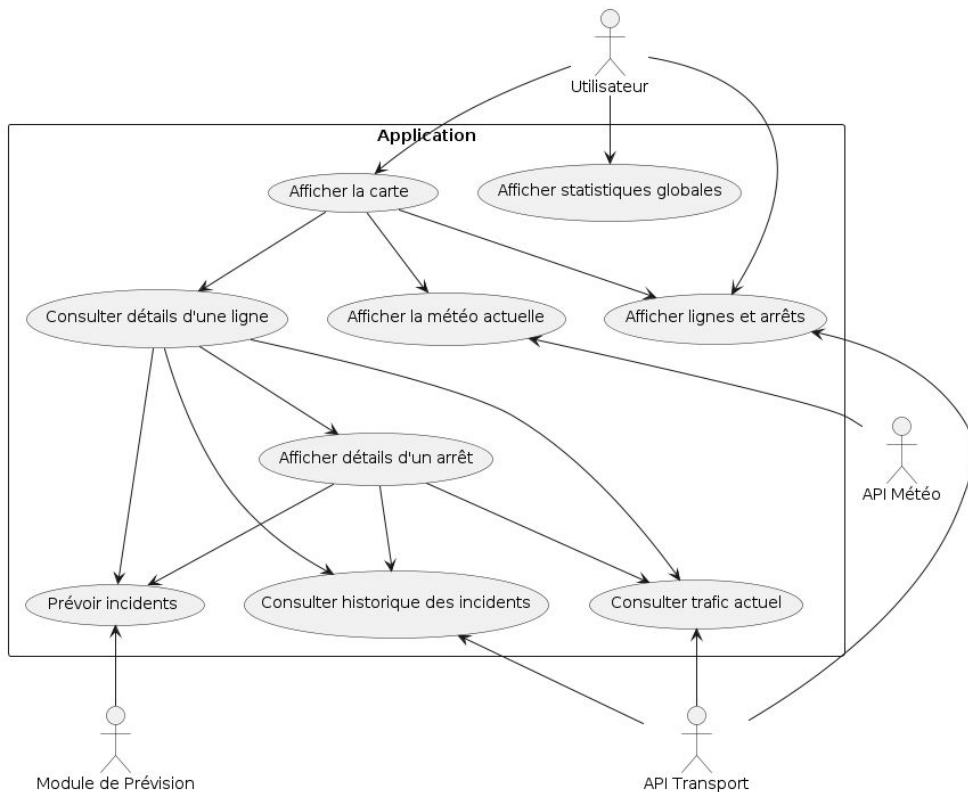
Fonctionnalités

Listes des fonctionnalités par ordre de priorité:

Fonctionnalité	Priorité
Carte interactive des lignes et arrêts	1
Prédiction de retards	2
Statistiques réseau et par ligne	3
Météo actuelle	4
État du trafic et incidents	5

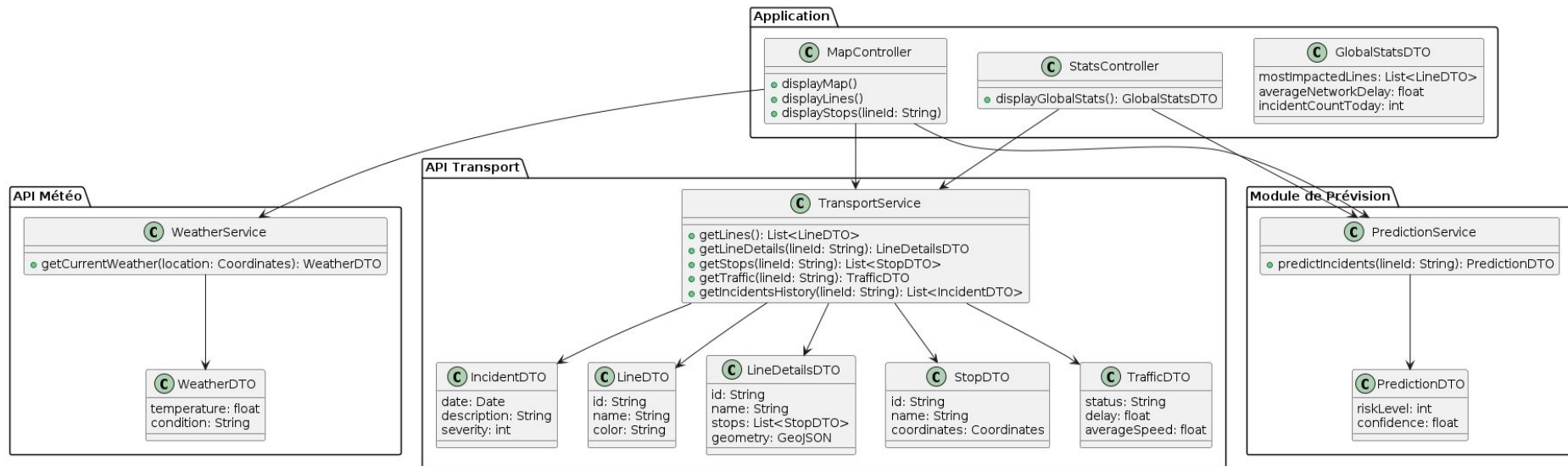
Diagramme de Cas d'utilisation

Diagramme de Cas d'Utilisation - Système d'Information Transport + Météo



Modélisation & API

Diagramme de Classes - Architecture API



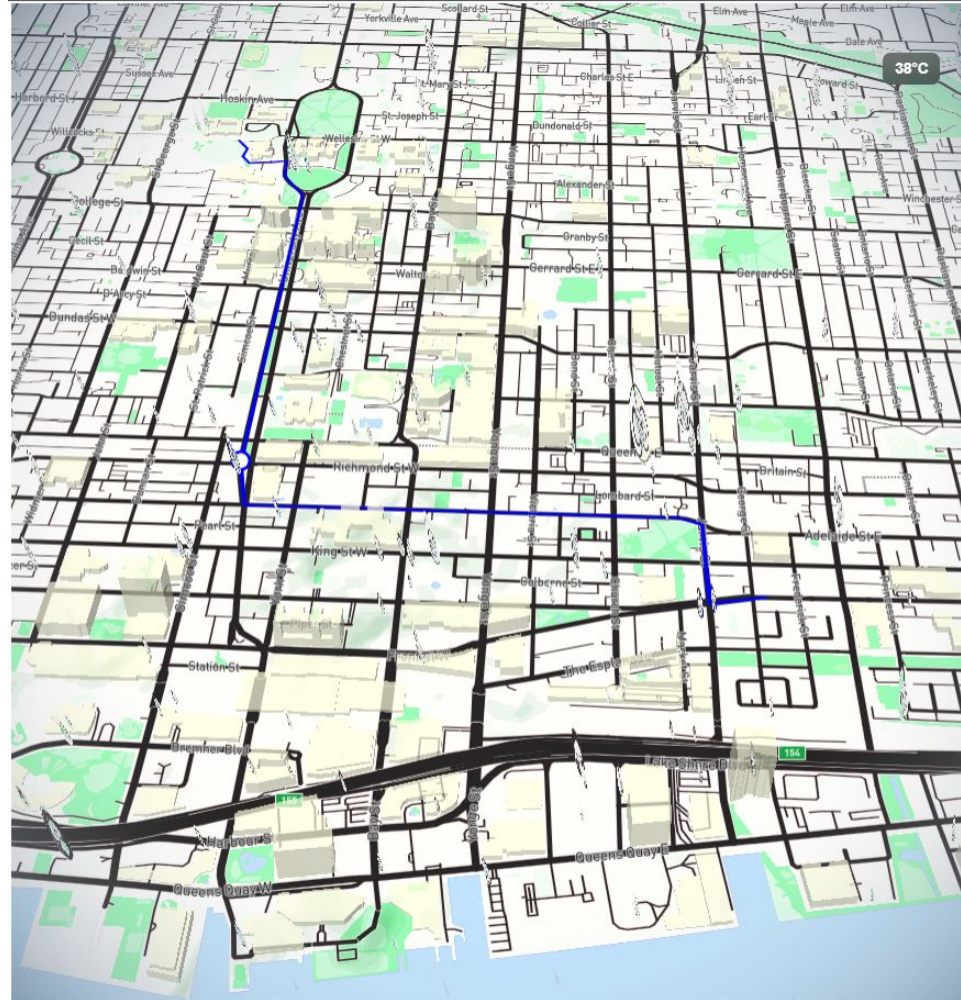
Modélisation & API

API REST

GET	/weather/current	Récupérer la météo actuelle	▼
GET	/lines	Récupérer toutes les lignes du réseau	▼
GET	/lines/{lineId}	Détails d'une ligne	▼
GET	/lines/{lineId}/traffic	Récupérer l'état du trafic pour une ligne	▼
GET	/lines/{lineId}/incidents	Historique des incidents d'une ligne	▼
GET	/lines/{lineId}/prediction	Prévision des incidents	▼
GET	/stats/global	Statistiques globales du réseau	▼

POC

- React + Mapbox GL JS.
- Affichage d'une lignes et ses arrêts
- Carte personnalisée orientée routes.
- Affichage de la météo.
- Validation des choix techniques.



Création des routes avec les données GTFS

Les données du réseau proviennent du format standard GTFS.

Ce format décrit les lignes, arrêts, trajets et tracés géographiques.

Il constitue la base de toutes les données affichées sur la carte.

Extraction et Structuration :

Les fichiers GTFS (routes, trips, stops, stop_times, shapes) sont analysés par un script Python.

Seules les lignes utiles sont conservées grâce à un filtrage préalable.

Les arrêts dupliqués sont détectés et fusionnés selon leurs coordonnées GPS.

Pour chaque ligne, le trajet le plus représentatif est sélectionné.

Création des routes avec les données GTFS

Les données sont normalisées en deux fichiers distincts : lignes et arrêts.

Les tracés sont exportés au format GeoJSON pour l'affichage Mapbox.

Ce format final est optimisé pour le chargement et le rendu côté frontend.

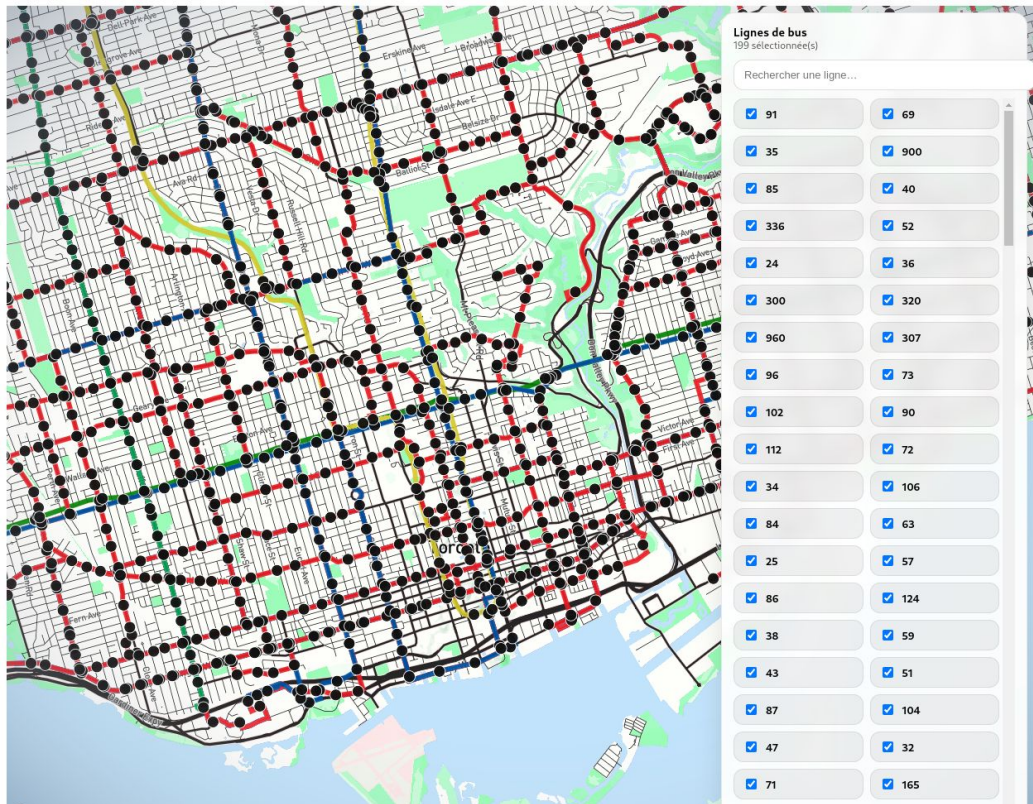
```
{
  "lignes": [
    {
      "id": 7,
      "nom": "7",
      "couleur": "EE3124",
      "long_nom": "Bathurst",
      "stopIds": [110, 3016, 3017, ...],
      "geometries": [
        [-79.41139, 43.64203],
        [-79.41141, 43.64215],
        ...
      ]
    }
  ]
}
```


Problème de performances

Un couple source + layer Mapbox était créé pour chaque ligne, et souvent pour chaque groupe d'arrêts.

Chaque ligne correspondait également à un composant React dédié.

Chaque arrêt correspondait également à un autre composant React



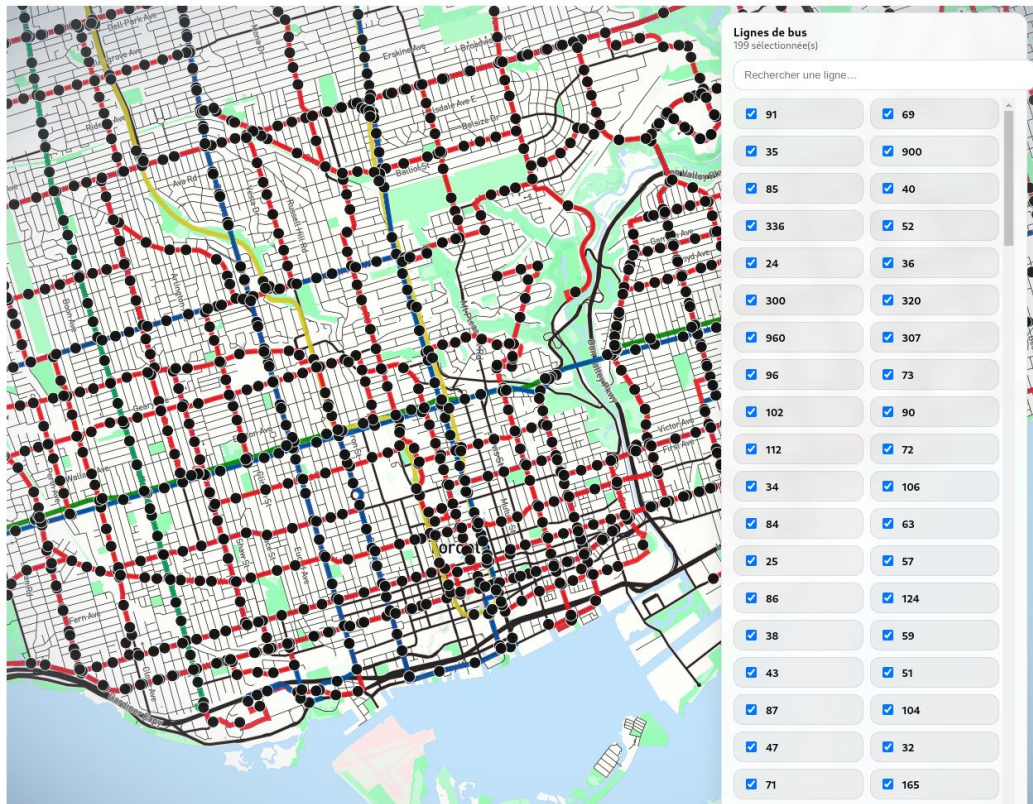
Solution mise en place

React gère uniquement l'état applicatif et les interactions utilisateur.

Mapbox GL est responsable exclusivement du rendu cartographique.

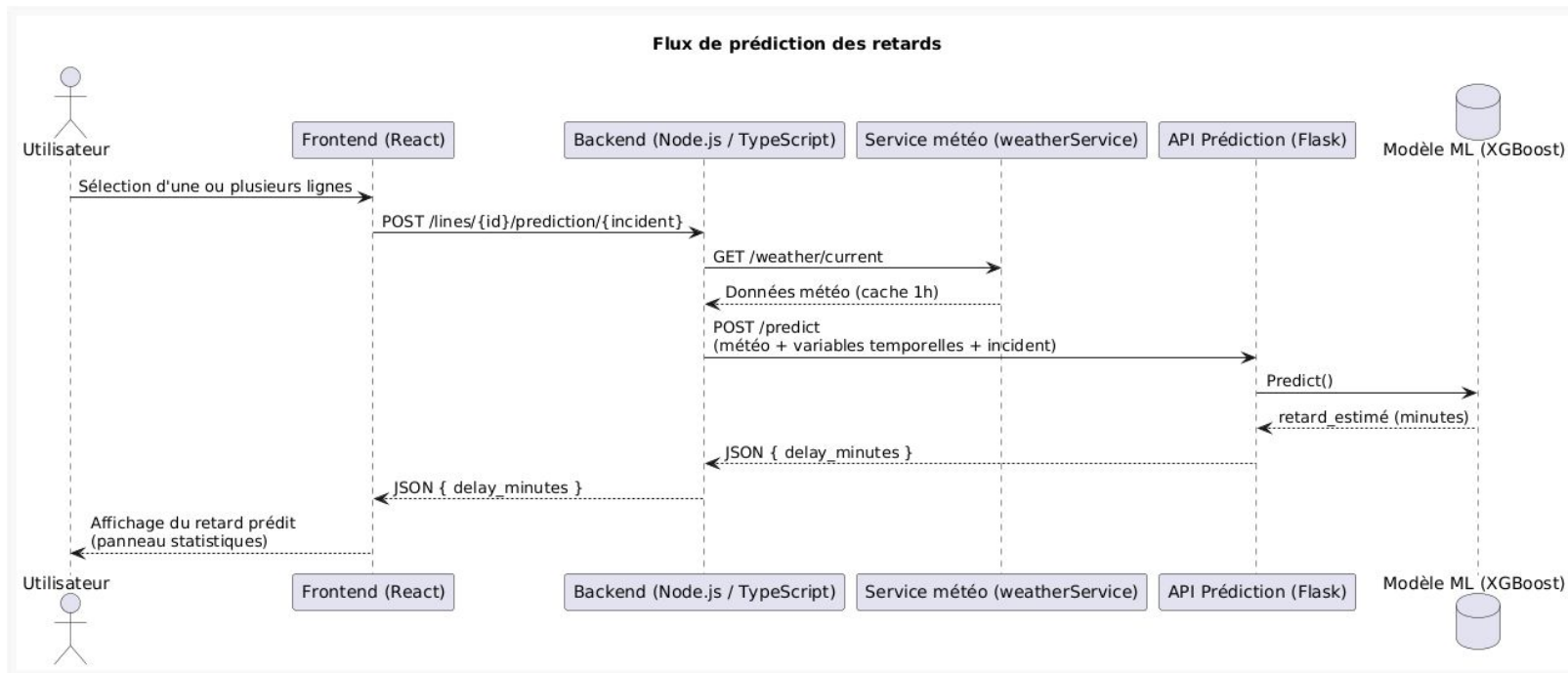
Toutes les lignes sont regroupées dans **une seule source** GeoJSON et affichées via un **unique** layer.

Tous les arrêts sont regroupés dans **une seconde source** GeoJSON avec un **seul** layer dédié.



Système de prédiction des retards

Le modèle est exposé via une API Python Flask, consommée par le back-end Node.js et intégrée au front-end React.

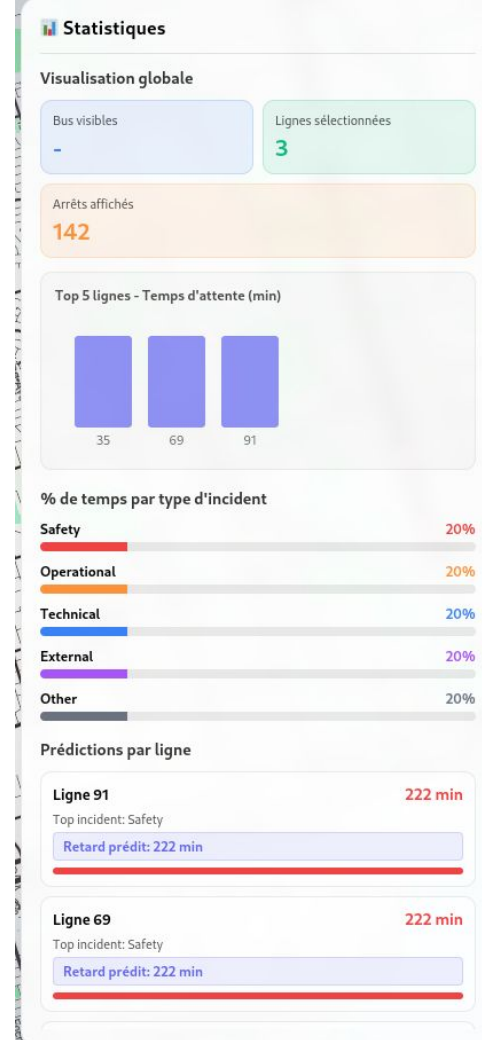


Système de prédiction des retards

Lorsque l'utilisateur sélectionne une ligne, l'application récupère les données météorologiques actuelles et envoie une requête de prédiction au back-end.

Celui-ci communique avec le service Python, obtient une estimation du retard et renvoie le résultat au front-end.

Les prédictions sont ensuite affichées de manière claire dans le panneau de statistiques.



Service météorologique

Un service météorologique dédié permet de récupérer des données réelles via une API externe.

Ces données sont mises en cache côté back-end afin de limiter les appels réseau et d'améliorer les performances.

Le format des données est adapté spécifiquement aux besoins du système de prédiction, garantissant une cohérence entre les différents services.

Manque de données sur :

- Dew_point_temp
- Humidex
- Visibility
- Station_pressure

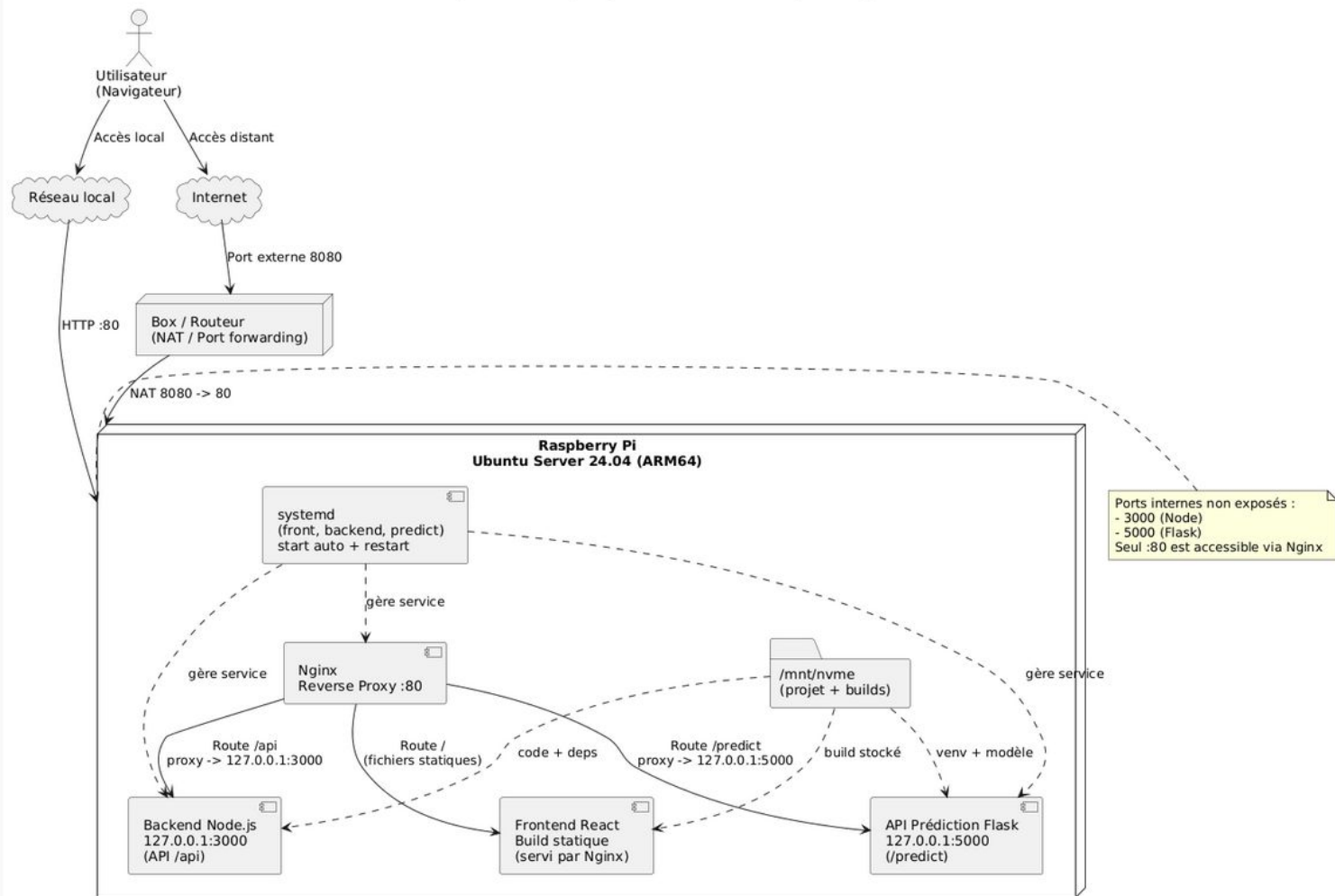
```
export type WeatherData = {  
  TEMP: number;  
  DEW_POINT_TEMP: number;  
  HUMIDEX: number | null;  
  PRECIP_AMOUNT: number;  
  RELATIVE_HUMIDITY: number;  
  STATION_PRESSURE: number;  
  VISIBILITY: number;  
  WEATHER_ENG_DESC: string;  
  WIND_DIRECTION: number | null;  
  WIND_SPEED: number | null;  
  LOCAL_DATE: string;  
  LOCAL_TIME: string;  
  WEEK_DAY: string;  
  LOCAL_MONTH: number;  
  LOCAL_DAY: number;  
}
```

Déploiement sur Raspberry Pi

L'ensemble de l'application a été déployé sur un Raspberry Pi afin de valider la faisabilité sur une plateforme embarquée.

Le front-end est servi de manière statique via Nginx, tandis que le back-end Node.js et l'API de prédiction Python communiquent uniquement en local.

Nginx agit comme point d'entrée unique, ce qui renforce la sécurité et permet un accès identique depuis le réseau local ou Internet.



Conclusion

Conclusion

Ce que nous avons réussi :

- Création d'un jeu de données
- Etude des données
- Une application web hébergée
- La mise en place de services multiplateforme
- Exploitation du modèle Python

Ce que nous avons raté :

- Modèles dysfonctionnels
- Pas assez d'expérimentation sur les données et leur prétraitement
- Récupération partielle des données météorologique
- Affichage du trafic en temps réel
- Statistiques historiques des lignes

Conclusion

Idées de poursuites

- Création de nouvelles variables dans le JDD
- Tester de prédire le délai sans les routes ou les incidents
- Tester de prédire l'incident plutôt que le délai
- Tester plus extensivement de retirer des outliers dans la variable cible
- Compléter la récupération des données météorologique dans l'application web
- Récupération et affichage de l'historique des incidents d'une ligne

Merci pour votre attention

Passage à la démo et aux questions

Adresse ip : 84.97.103.65:8080